



Protect Your SPFx Solutions: Build Secure Backends with Azure Functions

Don Kirkham - Microsoft MVP, M365 Development & Microsoft Graph



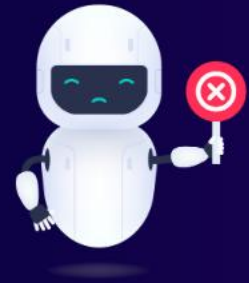
PHOTOGRAPHY DURING THE SESSION

Feel free to capture the moment! Taking pictures of the presentation, during the session is perfectly fine.



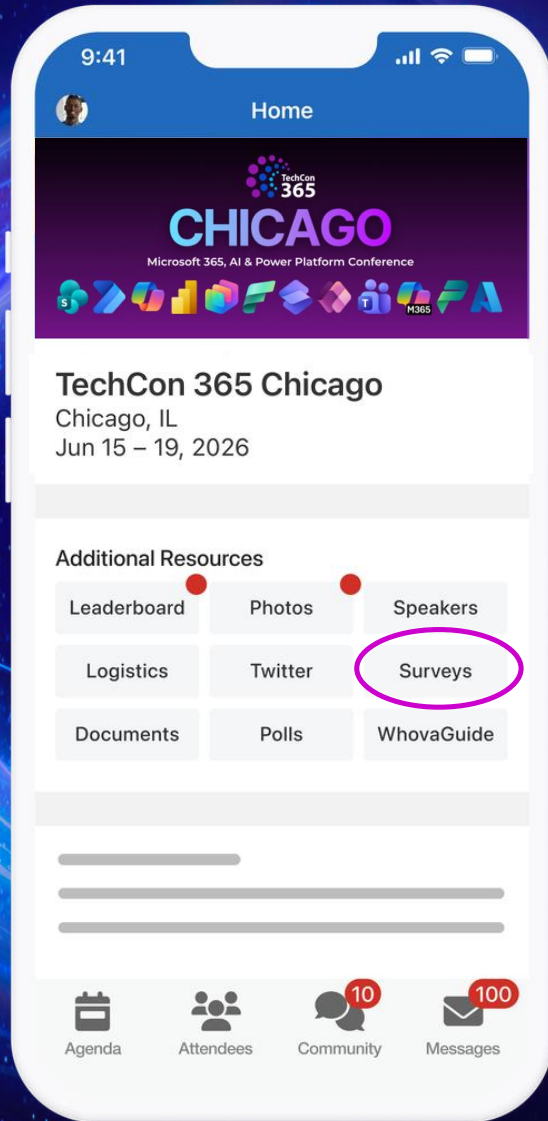
SELFIES WITH THE SPEAKER

We encourage interaction! If you'd like to take a selfie with the speaker, don't hesitate to ask. Most speakers will be happy to oblige during appropriate breaks.



RECORDING AND LIVE STREAMING

Recording or live streaming the session, in part or in full, is strictly prohibited. Thank you for respecting our content and Speakers.



How was the session?

Search for *Ukhova* in the
App Store or Google Play



Fill out the Session Surveys in the
TechCon 365 Chicago Event App
and be eligible to **WIN PRIZES!**

DON KIRKHAM

MICROSOFT MVP, M365 DEVELOPMENT & MICROSOFT GRAPH



Arlington, Texas, USA



Senior Technology Advisor



<https://donkirkham.com>



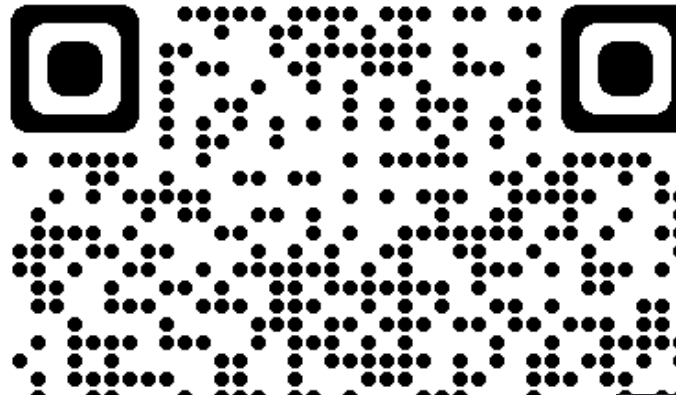
@DonKirkham



/in/DonKirkham

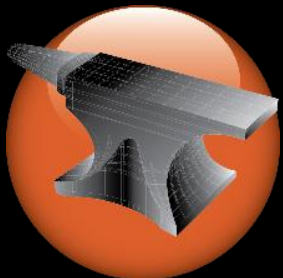


DonKirkham



Microsoft®
Most Valuable
Professional





SOLUTIONFOUNDRY®

M365 Solutions and Beyond. Built for your needs. By our experts.

Secure Microsoft 365 modernization, automation, and custom development—delivered by our team of senior consultants, developers, and Microsoft MVPs

Core Microsoft 365 services and custom tools

- SPFx Development – SharePoint Framework solutions that extend what M365 can do out of the box.
- SharePoint Modernization – Plan, migrate and transition from Classic SharePoint to the Modern experience.
- Power Platform– Data Visualization with Power BI, workflow automation with Power Automate, and user-friendly Power Apps.
- AI & Copilot Integration – Copilot and other AI tools design and implementation.

Federal & Regulated Program Support

- Decades of experience supporting federal and highly regulated organizations with compliance-driven systems and mission support solutions.
- Primary architects and implementers of enterprise portals, dashboards, and knowledge platforms that standardize processes, improve visibility, and streamline reporting.

General Qualifications

- 3 decades of federal contracting experience
- Microsoft MVP expertise and industry recognition
- Proven ability to embed with customer teams as trusted advisors
- Deep understanding of federal compliance, security, and accountability requirements
- Vast experience with business process, data management and communications analysis, improvement and automation.

Representative Solution Types

- Enterprise portals, dashboards, and reporting for operational visibility
- Compliance workflows, document management, and audit readiness solutions
- Knowledge platforms and self-service support content organized by mission area

www.solutionfoundry.com



PICTURE TIME!

Thursday June 26, 2025
1:33 PM
Room 344

Today's Activities

- 12:40 PM - 01:50 PM
Power BI Storage Models: The Ultimate Showdown - Philip Seamark
- 02:00 PM - 03:10 PM
Enhancing SPFx projects with Live Data integration to SharePoint, MS Graph and beyond - Don Kirkham
- 04:00 PM - 05:10 PM
SSMS 21 Spotlight: What's new and why it matters - Makena Barickman

04:00 PM - 05:10 PM
Enhancing SPFx projects with Live Data integration to SharePoint, MS Graph and beyond - Don Kirkham

04:00 PM - 05:10 PM
SSMS 21 Spotlight: What's new and why it matters - Makena Barickman

The Problem

Your SPFx web part reads directly from a SharePoint list.

But not every user who can see the page has permission to that list — and the web part has no identity of its own. It can only act as the signed-in user.



THE CONSEQUENCE

Users without access hit a wall

403 errors, empty lists, or blank cards — the experience silently breaks for a subset of people.

Loosening list permissions is risky

Granting everyone direct access over-shares data and removes your control over who reads what.



DEMO!

Session Roadmap

01

Azure Functions Orientation

What they are, how they work, local dev loop

02

PnPjs in Node.js

App-only auth, SPFI instance, list operations

03

Tier 1: Origin-Validated API

Lightweight protection + CRUD endpoints

04

Tier 2: Entra ID Protected API

Custom app registration + token validation

05

Calling Both APIs from SPFx

HttpClient vs. AadHttpClient side by side

06

Deploy & Harden

Key Vault, App Insights, next steps

Azure Functions Orientation

What they are · How they work · Local dev loop



What Are Azure Functions?



Serverless Compute

Run code on demand — no servers to manage. Azure handles scaling and patching.



Triggers

- HTTP request
- Timer / CRON schedule
- Queue & Service Bus
- Blob & Event Grid



Pricing

- **Consumption** — PAYG, cold starts
- **Flex Consumption** — warm starts
- **Premium** — no cold starts
- **Dedicated** — flat rate



Languages

- C# / .NET
- JavaScript / TypeScript
- Python
- Java
- PowerShell

Our Picks for SPFx

HTTP request trigger

+

Flex Consumption

+

TypeScript hosted in Node.js

On Demand, Pay-for-use, and one language across our SPFx web part and API.

The HTTP Trigger – Anatomy of a Function

TypeScript

```
import {
  app,
  HttpRequest,
  HttpResponseInit,
  InvocationContext
} from '@azure/functions';

app.http('getItems', {
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  authLevel: 'anonymous',
  handler: async (req, ctx) => {
    // your logic here
    return {
      status: 200,
      body: JSON.stringify(data)
    };
  }
});
```

authLevel: 'anonymous'

We handle auth ourselves — don't let Azure add its own key requirement on top.

methods: ['GET', 'POST', 'PATCH', 'DELETE']

Declare which verbs this endpoint accepts. This API registers ONE function per auth tier and lets the HTTP method pick the CRUD action (GET=read, POST=create, PATCH=update, DELETE=delete).

handler async function

All Azure Function logic lives here. Async/await throughout — PnPjs calls are all promises.

Project Structure & Local Dev Loop

Project Structure

```
apiDemo/  
├── src/  
│   ├── functions/  
│   │   ├── domainRestricted.ts  
│   │   ├── entraProtected.ts  
│   │   ├── health.ts  
│   │   └── helpers.ts  
│   ├── security/  
│   │   ├── domainRestriction.ts  
│   │   └── entraAuth.ts  
│   ├── sharepointClient.ts  
│   ├── config.ts  
│   └── httpError.ts  
├── infra/main.bicep  
├── tests/  
├── host.json  
├── local.settings.json  
└── package.json
```

1

VS Code + Functions Extension

Scaffolds, runs, and debugs your app locally with breakpoints.

2

local.settings.json

- Secrets & config — maps to Azure App Settings
- CORS origins set under Host.CORS section
- **Never commit this file**

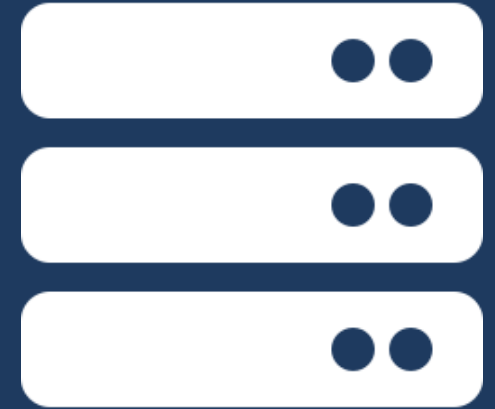
3

func start

Spins up the local HTTP server — test each function URL with Postman before touching SPFx.

PnPjs in a Node.js Context

App-only auth · SPFI setup · Shared instance pattern



PnPjs: Browser vs. Node.js

In Your SPFx Web Part

- Uses @pnp/sp with spfi()
- PageContext provides auth context
- Runs in the browser — user's session token
- SharePoint auth is implicit
- @pnp/sp/presets/all or selective imports

In Your Azure Function

- Same @pnp/sp package — same API surface
- No browser, no page context
- Must provide auth explicitly
- Uses AzureIdentity (@pnp/azidjsclient) over SPDefault()
- App identity — certificate (client secret is rejected by SharePoint REST)
- Managed Identity also works against SharePoint — a passwordless option; this project uses a certificate



Same API calls you already know — just a different auth setup underneath

App-Only Auth: Certificate Setup



Register App in Entra ID

New app registration. No redirect URIs needed — this is a service/daemon app, not a user-facing client.



Add API Permissions

Microsoft Graph or SharePoint: Sites.Selected (recommended) or Sites.ReadWrite.All. Admin consent required.



Generate & Upload Certificate

SharePoint REST only honors certificate-based app-only tokens — a client secret is rejected. Create and upload a cert, then put the .pem in the SHAREPOINT_CERT_PEM app setting and store it in Key Vault.

src/sharepointClient.ts – Shared SPFI Instance

```
import { spfi } from '@pnp/sp';
import { SPDefault } from '@pnp/nodejs';
import { AzureIdentity } from '@pnp/azidjsclient';
import { ClientCertificateCredential } from '@azure/identity';

const credential = new ClientCertificateCredential(
  process.env.AZURE_TENANT_ID,
  process.env.SHAREPOINT_APP_CLIENT_ID,
  { certificate: process.env.SHAREPOINT_CERT_PEM });

export const sp = spfi(process.env.SHAREPOINT_SITE_URL)
  .using(SPDefault(), AzureIdentity(credential, [process.env.SHAREPOINT_RESOURCE]));
```

DEMO!

Tier 1: Origin-Validated API

Lightweight protection + SharePoint CRUD via PnPjs



What Is Origin Validation?



What it DOES protect

- Browsers automatically send the Origin header
- Validates the caller is a known SharePoint domain
- Rejects unknown/missing origins with HTTP 403
- Good guard against casual misuse or accidental exposure
- Zero friction for legitimate SPFx callers



What it DOESN'T protect

- Origin headers can be spoofed by non-browser clients
- Any server-side tool (Postman, curl) can fake origin
- Provides NO identity verification
- Not appropriate for sensitive or write-heavy operations
- **Not a substitute for authentication**

When Is This Tier Appropriate?

Read-heavy endpoints that surface non-sensitive SharePoint list data.

Internal tools where the consumer audience is known and trusted.

Stepping stone — add Entra ID when requirements demand it.

Tier 1: CRUD Endpoints with PnPjs

All four verbs hit one route — `/api/domain/{itemId?}` — dispatched by HTTP method.

GET

Get Items

GET `/api/domain`

```
sp.web.lists.getByTitle('Requests')
  .items.select('Id','Title','Status')()
```

POST

Create Item

POST `/api/domain`

```
sp.web.lists.getByTitle('Requests')
  .items.add({ Title: body.title, Status: 'New' })
```

PATCH

Update Item

PATCH `/api/domain/{itemId}`

```
sp.web.lists.getByTitle('Requests')
  .items.getById(id).update({ Status: body.status })
```

DELETE

Delete Item

DELETE `/api/domain/{itemId}`

```
sp.web.lists.getByTitle('Requests')
  .items.getById(id).delete()
```

Origin Validation Middleware

src/security/domainRestriction.ts

```
const ALLOWED = (process.env.ALLOWED_CALLER_DOMAINS ?? '')
    .split(',')
    .map(o => o.trim())
    .filter(Boolean);

export function ensureAllowedDomain(req: HttpRequest): void {
    const origin = req.headers.get('origin')
        ?? (req.headers.get('referer')
            ? new URL(req.headers.get('referer')!).origin
            : null);
    if (!origin) throw httpError(403, 'Request origin or referer is required.');
```

```
    if (!ALLOWED.includes(origin)) throw httpError(403, 'Caller domain is not allowed.');
```

```
}
```

Environment Variable Allowlist

ALLOWED_CALLER_DOMAINS =
https://contoso.sharepoint.com,...
Comma-separated — no secrets in code.

Check Both Headers

Origin is set by CORS requests. Referer is set by navigation. Checking both handles edge cases.

Throw error if not allowed

On a missing or not allowed origin, it throws `httpError(403)`, which is caught by the global `runWithErrorHandling` handler.

Usage in Handler

```
ensureAllowedDomain(req);
```

Interrupts execution with error; otherwise, safe to continue

DEMO!

Tier 2: Entra ID Protected API

Custom app registration · Token validation · Identity-aware



When to Step Up to Entra ID Auth



Sensitive Data

PII, financial records, or data that shouldn't be accessible to anonymous API consumers.



User Identity Matters

You need to know WHO is calling — for audit trails, row-level permissions, or personalized data.



Write Operations

Creating, updating, or deleting data carries more risk. Knowing the caller's identity is essential.



Non-Intranet Callers

If the API could ever be called from outside your SharePoint domain, origin validation alone isn't sufficient.

Tier 1 and Tier 2 can coexist — use the right tier for each endpoint's risk profile

Custom App Registration: Expose an API

1

New App Registration

Entra ID → App registrations → New registration. Name it something recognizable (e.g. SPFx-SecureAPI-Server). No redirect URI.

2

Expose an API

Set Application ID URI (api://{client-id}). Add a scope — e.g. SharePoint.ReadWrite. Set to 'Admins and users'.

3

Register Client App

Separate app registration for your SPFx solution. This is the CALLER — it requests the scope you defined above.

4

Grant API Permission

In the CLIENT app: API permissions → Add → My APIs → select your server app → check the scope. Admin consent.

Token Validation: What Claims Matter?

iss

Issuer

Must be your tenant's Entra ID endpoint. Rejects tokens from other tenants.

aud

Audience

Must match your server app's Application ID URI. Tokens meant for Graph won't pass.

azp / appid

App ID

The client app that requested the token. Enforced against an ALLOWED_CLIENT_APP_IDS allowlist — must be YOUR SPFx app registration.

scp

Scope (not enforced)

This project does NOT check scp today — it relies on the app-ID allowlist. Validating scp too is a reasonable hardening add.

```
src/security/entraAuth.ts – validateEntraToken()
```

```
import jwt from 'jsonwebtoken'; import jwksClient from 'jwks-rsa';
export async function validateEntraToken(req: HttpRequest): Promise<JwtPayload> {
  const token = req.headers.get('authorization')?.match(/^Bearer\s+(.+)$/i)?.[1];
  if (!token) throw httpError(401, 'Authorization bearer token is required.');
```

// jwt.verify with { algorithms:['RS256'], audience, issuer } + jwks-rsa signing key,
// then check azp/appid against ALLOWED_CLIENT_APP_IDS. Returns the JwtPayload.

```
}
```

DEMO!

Calling Both APIs from SPFx

[HttpClient](#) vs. [AadHttpClient](#) · [webApiPermissionRequests](#)



Tier 1 from SPFx: HttpClient

HttpClient is the standard SPFx HTTP client — no token, no extra config. The browser sends the Origin header automatically. You just call your API URL.

MyWebPart.ts

```
const client = this.context.httpClient;

const response = await client.get(
  process.env.
  API_BASE_URL + '/api/domain',
  HttpClient.configurations.v1
);

if (!response.ok) {
  // Surface 403 to user
  throw new Error(`API error: ${response.status}`);
}

const items = await response.json();
```

No Token Needed

HttpClient sends the browser Origin header automatically. Your origin validation middleware handles the rest.

Store URL in Config

Don't hardcode the function URL. Use web part properties or an environment config so the URL can change per environment.

Error Handling

A 403 from origin rejection should surface a meaningful message — not a generic network error.

Tier 2 from SPFx: AadHttpClient

Step 1: Declare the permission in your web part manifest

```
"webApiPermissionRequests": [ { "resource": "api://{client-id}", "scope": "SharePoint.ReadWrite" } ]
```

MyWebPart.ts – AadHttpClient call

```
const aadClient = await this.context
  .aadHttpClientFactory
  .getClient(
    'api://{your-api-client-id}');

const response = await aadClient.get(
  process.env.
  SECURE_API_URL + '/api/entra',
  AadHttpClient.configurations.v1
);

if (!response.ok) {

  // 401 = bad/expired token
  throw new Error(`Auth error: ${response.status}`);
}
```

Admin Approval Required

After deploying your solution package, a SharePoint admin must approve the API permission in the SharePoint Admin Center.

SPFx Handles the Token

AadHttpClientFactory acquires and caches the Bearer token automatically. You never touch the token directly.

getClient = Resource URI

Pass the Application ID URI (api://...) of your SERVER app registration — not the client app.

Side-by-Side: What Actually Changes

Tier 1 – HttpClient		Tier 2 – AadHttpClient	
Import	<code>import { HttpClient } from '@microsoft/sp-http'</code>	<code>import { AadHttpClient } from '@microsoft/sp-http'</code>	
Get client	<code>this.context.httpClient</code>	<code>await this.context.aadHttpClientFactory.getClient('api://{client-id}')</code>	
Make call	<code>client.get(url, HttpClient.configurations.v1)</code>	<code>aadClient.get(url, AadHttpClient.configurations.v1)</code>	
Token	None – browser sends Origin	Auto-acquired Bearer token in Authorization header	
Manifest	No changes needed	webApiPermissionRequests entry required	
Admin step	None	Admin approves permission in SPO Admin Center	

DEMO!

Deploy & Harden

Key Vault · App Insights · Next steps



Production Hardening Checklist



Key Vault for Secrets

Never store CLIENT_SECRET in App Settings as plain text. Use a Key Vault reference: @Microsoft.KeyVault(SecretUri=...). Zero code changes required.



Certificates: Time Them

This API uses certificate auth. Certificates expire, so set calendar reminders for expiry and rotation. Expired credential = production outage.



Application Insights

One checkbox in the Azure Portal. Gets you request traces, failures, response times, and custom telemetry. Turn it on always.



Managed Identity (Next Level)

Replace certificate entirely. Assign a Managed Identity to your Function App and grant it SharePoint access. No secrets, no certificates.



Automate Deployment (CI/CD)

Ship through GitHub Actions or Azure DevOps Pipelines, not manual publish. Run two pipelines: one for infrastructure (Bicep) and a separate one for application code, so each can deploy and roll back independently.



CORS — Lock It Down

In local.settings.json: allow localhost. In Azure Function App: only add your SharePoint domains. Don't use wildcard *.

Recap

- Azure Functions: serverless HTTP backends for SPFx
- PnPjs works the same in Node — just needs explicit auth
- Tier 1: origin validation for low-risk read endpoints
- Tier 2: Entra ID + token validation for sensitive ops
- HttpClient vs. AadHttpClient — one line of difference
- Production Ready: Key Vault, App Insights, CI/CD deploy

Resources

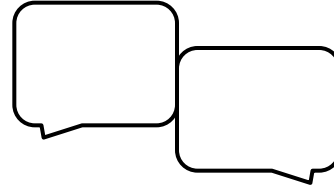
Azure Functions Repo: github.com/donkirkham/apiDemo

Data Demo Repo: github.com/donkirkham/dataDemo

Blog: donkirkham.com

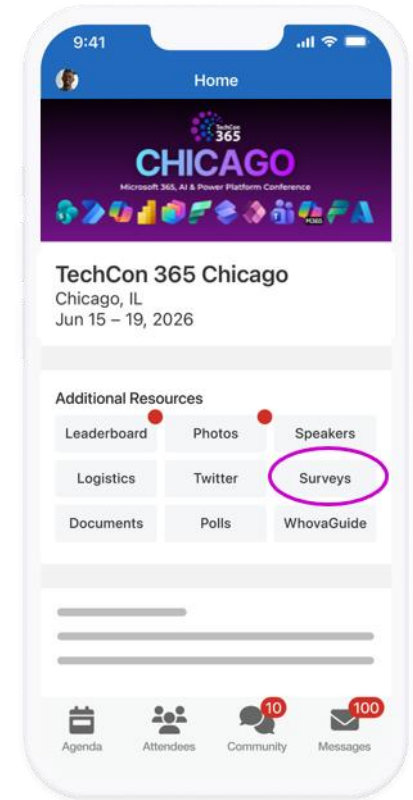
Twitter/X: [@donkirkham](https://twitter.com/donkirkham)





Q&A

Questions?



Don Kirkham



<https://donkirkham.com>



@DonKirkham



/in/DonKirkham



github.com/DonKirkham/Presentations

Whova

Please rate my session

- Open the Whova app
- Surveys → fill it out

Be eligible to **win prizes!**